

OBJECT ORIENTED PROGRAMMING

PROJECT REPORT

Keylogger Application

An Educational Cybersecurity Project Using OOP in C++

Table of Contents

1. Introduction	3
2. Project Background and Purpose	4
3. What is a Keylogger?	5
4. OOP Concepts Used in This Project	6
5. System Design and Architecture	7
6. Code Explanation	9
7. How the Program Works (Step by Step)	11
8. Windows API Functions Used	12
9. Limitations and Security Concerns	13
10. Possible Improvements	14
11. Conclusion	15

1. Introduction

This report is written to explain a keylogger application that was built as a semester project for the Object Oriented Programming (OOP) course. The project was done as part of a Cybersecurity degree program in the second semester. The main goal of this project is to understand how OOP works in a real-world application and to learn how basic system-level programs are made using C++.

The keylogger application was written in C++ using the Windows API. It uses important OOP concepts like classes, objects, encapsulation, abstraction, and static members. These are core ideas taught in OOP and this project shows how they can be applied in an actual program.

It is important to mention that this project is made strictly for educational purposes. The goal is not to misuse the program but to understand how such software works at the code level. Understanding how keyloggers work is an important part of cybersecurity education, because in order to defend against a threat, you need to understand how it operates.

This report will walk through all the parts of the program, explain the OOP concepts that were used, and show how the program runs from start to finish. The report also covers the design of the software, the Windows API functions used, and the possible improvements that could be made in the future.

2. Project Background and Purpose

2.1 Why This Project?

In the second semester of the Cybersecurity program, students are required to complete an OOP project using C++. The project has to show a good understanding of OOP principles and should be something meaningful related to the field of study. Since the degree is in Cybersecurity, a keylogger was chosen as the project topic because it is directly related to the subject.

Cybersecurity professionals need to know about malicious tools like keyloggers so they can:

- Detect when a keylogger is running on a system
- Build antivirus or monitoring software that can identify such programs
- Understand what data could be at risk if a system is infected
- Write better and more secure software by knowing what threats exist

2.2 Project Goals

The following goals were set before starting this project:

- Build a working keylogger using C++ and OOP principles
- Use a class structure to organize the code properly
- Apply concepts like encapsulation, abstraction, and static members
- Use the Windows API to capture keystrokes in real time
- Log the captured keystrokes to a text file
- Write clean, readable code with proper structure

2.3 Tools and Technologies Used

Tool / Technology	Purpose
C++ Language	Main programming language used to write the program
Windows API (Win32)	Used to install the keyboard hook and capture system-level keystrokes
Visual Studio / g++	IDE or compiler used to build and run the project
ofstream (C++ STL)	Used to write the captured key data to a log file
Windows.h Header	Provides access to all Windows API functions and data types

3. What is a Keylogger?

3.1 Basic Definition

A keylogger (also called a keystroke logger) is a type of software that records every key pressed on a keyboard. Every time a user presses a key, the keylogger captures it and saves it somewhere, like a text file. The user usually does not know that their keystrokes are being recorded.

Keyloggers can be either hardware-based (a small physical device plugged into the keyboard port) or software-based (a program running in the background). This project is a software-based keylogger written in C++.

3.2 How Keyloggers Are Used in the Real World

Keyloggers are used in two very different ways:

Malicious Use (Bad):

- Stealing passwords and bank login details
- Spying on private messages
- Gathering personal information without consent
- Corporate espionage

Legitimate Use (Good):

- Parental control software to monitor children's online activity
- Employee monitoring in companies (with consent)
- Law enforcement investigations (with legal permission)
- Security research and penetration testing
- Recovering lost text when an application crashes

3.3 How a Software Keylogger Works

A software keylogger works by hooking into the operating system at a low level. In Windows, this is done using the `SetWindowsHookEx` function, which installs a hook — a special callback function that the operating system calls every time a key is pressed. This is called a low-level keyboard hook.

When the hook is triggered, the callback function receives information about which key was pressed. The keylogger then processes this information and saves it to a file. This happens invisibly in the background while the message loop keeps the program running.

4. OOP Concepts Used in This Project

This project was built with a strong focus on Object Oriented Programming. Below are the main OOP concepts that were applied in the code, along with a simple explanation of each one.

OOP Concept	Where Used	Explanation
Class	KeyLogger class	All the data and functions related to the keylogger are grouped inside one class called KeyLogger.
Object	KeyLogger logger (in main)	An object called 'logger' is created from the KeyLogger class. It is used to call all the methods.
Encapsulation	Private section of KeyLogger	The keyboard hook handle and the logKeystroke() function are kept private so outside code cannot access or misuse them.
Abstraction	start(), run(), stop() methods	The user of the class only needs to call three simple functions. The complex internals are hidden away.
Static Member (Variable)	static KeyLogger* instance	A static pointer is used so the static callback function can access the class instance without an object reference.
Static Member (Function)	static LRESULT CALLBACK KeyboardProc()	The Windows API requires a static callback function. It is declared static inside the class and uses the static instance pointer.
Constructor	KeyLogger()	The constructor initializes the keyboardHook to NULL and sets the static instance pointer to the current object.

4.1 Encapsulation in Detail

Encapsulation means putting related data and functions inside a class and hiding the ones that should not be accessed from outside. In this project, the keyboard hook handle (keyboardHook) and the logKeystroke() function are both in the private section of the KeyLogger class. This means no code outside the class can directly touch these. Only the class's own functions can use them. This is good practice because it prevents accidental misuse.

4.2 Abstraction in Detail

Abstraction means showing only what is necessary and hiding the complexity. The main() function only needs to do three things: call start(), call run(), and call stop(). It does not need to know anything about

hooks, Windows API calls, or file operations. All of that is hidden inside the class. This makes the code easier to read and use.

4.3 Static Members in Detail

Static members belong to the class itself, not to any single object. In this project, a static pointer called `instance` is used to store the address of the current `KeyLogger` object. This is needed because the Windows API requires the keyboard callback function to be a static function (it cannot be a normal member function). Since static functions cannot access non-static data directly, the static instance pointer is used as a bridge to call `logKeystroke()` on the actual object.

5. System Design and Architecture

5.1 Overview of the Class Structure

The program has one main class called KeyLogger. Below is the structure of the class:

```
class KeyLogger {
private:
    HHOOK keyboardHook;          // Handle to the keyboard hook
    void logKeystroke(int key);   // Logs a single key to file
    static LRESULT CALLBACK KeyboardProc(...); // Windows callback
    static KeyLogger* instance;  // Pointer to current object
public:
    KeyLogger();                 // Constructor
    bool start();                // Installs the keyboard hook
    void run();                  // Runs the Windows message loop
    void stop();                 // Removes the keyboard hook
};
```

5.2 How the Components Connect

The design is clean and modular. Each function has one specific job:

- start() installs the Windows hook and returns true if it succeeded
- run() keeps the program alive using a Windows message loop
- stop() cleans up by removing the hook when the program ends
- logKeystroke() handles the actual writing to the log file
- KeyboardProc() is the hook callback that Windows calls on every keypress

5.3 Data Flow Diagram

The following steps show how data flows through the program from a key press to the log file:

Step 1	User presses a key on the keyboard
Step 2	Windows detects the keypress and calls the registered hook callback (KeyboardProc)
Step 3	KeyboardProc checks if nCode >= 0 and if the event is WM_KEYDOWN
Step 4	It extracts the virtual key code from the KBDLLHOOKSTRUCT structure
Step 5	It uses the static instance pointer to call logKeystroke() on the KeyLogger object
Step 6	logKeystroke() opens the keylog.txt file in append mode

Step 7	The key is identified and the correct character or label is written to the file
Step 8	The file is closed and the hook passes control to the next hook in the chain

6. Code Explanation

6.1 The Constructor

The constructor is the first function that runs when a KeyLogger object is created. It does two things: it sets the keyboardHook to NULL (meaning no hook is installed yet), and it sets the static instance pointer to point to the current object. This is needed so the static callback can reach the object later.

```
KeyLogger() {  
    keyboardHook = NULL;  
    instance = this; // Save 'this' for the static callback to use  
}
```

6.2 The logKeystroke() Function

This is the function that actually saves key data to the file. It opens keylog.txt in append mode, checks which key was pressed using virtual key codes (VK_ constants), and writes the matching text. Special keys like Backspace, Enter, Tab, Shift, and Ctrl are written with descriptive labels like [BACKSPACE]. Letters are handled using their ASCII values. If Shift is pressed, the letter is uppercase; otherwise it is lowercase.

```
void logKeystroke(int key) {  
    std::ofstream logfile;  
    logfile.open("keylog.txt", std::ios::app);  
    if (!logfile.is_open()) return;  
  
    if (key == VK_BACK)    logfile << "[BACKSPACE]";  
    else if (key == VK_RETURN) logfile << "\n";  
    else if (key >= 0x41 && key <= 0x5A) { // A-Z  
        bool shiftPressed = GetAsyncKeyState(VK_SHIFT) & 0x8000;  
        logfile << (char)(shiftPressed ? key : key + 32);  
    }  
    // ... more key handling  
    logfile.close();  
}
```

6.3 The Static Callback Function

KeyboardProc is the callback that Windows calls every time a key event happens. It must be static because Windows API requires it. It checks if nCode is valid (≥ 0) and if the event is a key-down event (WM_KEYDOWN). Then it reads the virtual key code and calls logKeystroke through the static instance pointer. Finally, it always calls CallNextHookEx to pass the event to the next hook in the chain, which is important for the system to work correctly.

```
static LRESULT CALLBACK KeyboardProc(int nCode, WPARAM wParam, LPARAM lParam) {  
    if (nCode >= 0) {  
        if (wParam == WM_KEYDOWN) {  
            KBDLLHOOKSTRUCT* pKeyBoard = (KBDLLHOOKSTRUCT*)lParam;  
            instance->logKeystroke(pKeyBoard->vkCode);  
        }  
    }  
}
```

```
    }  
    return CallNextHookEx(NULL, nCode, wParam, lParam);  
}
```

6.4 The start() Function

The `start()` function installs the low-level keyboard hook using `SetWindowsHookEx`. It passes `WH_KEYBOARD_LL` (which means low-level keyboard hook), the callback function `KeyboardProc`, and the module handle. If the hook fails to install, it gets the error code using `GetLastError()` and returns false. If it works, it returns true.

6.5 The run() and stop() Functions

`run()` enters the Windows message loop. `GetMessage()` keeps the program running and delivers system messages. Without this loop, the program would exit immediately and the hook would stop working. `stop()` calls `UnhookWindowsHookEx` to remove the hook cleanly when the program is done.

7. How the Program Works (Step by Step)

This section walks through the complete execution of the program from the moment it is launched to when it is closed.

7.1 Program Startup

The main() function creates a KeyLogger object called logger. When this object is created, the constructor runs automatically and sets keyboardHook to NULL and instance to point to the logger object.

7.2 Hook Installation

logger.start() is called. This installs the keyboard hook using the Windows API. A success message is printed to the console and the program waits for keystrokes.

7.3 Capturing Keystrokes

logger.run() starts the message loop. Now, every time the user presses a key anywhere on the system, Windows calls KeyboardProc. This function identifies the key and calls logKeystroke() which writes the key to keylog.txt.

7.4 Program Shutdown

When the user presses Ctrl+C in the console window, the program receives a signal to stop. The message loop ends and logger.stop() is called, which removes the hook using UnhookWindowsHookEx. The program then exits cleanly.

7.5 The Log File

During the entire time the program runs, a file called keylog.txt is created in the same folder as the executable. Every key that is pressed is added to this file in order. For example, if someone types Hello World and presses Enter, the log file would contain: Hello World followed by a newline character.

```
// Sample output in keylog.txt
Hello World
password123
[BACKSPACE][BACKSPACE]
my email is example@gmail.com
```

8. Windows API Functions Used

This project makes heavy use of the Windows Application Programming Interface (Win32 API). Below is an explanation of each API function used and what it does.

API Function	Description
SetWindowsHookEx()	Installs a hook procedure to monitor keyboard input events at a low level. It takes the hook type (WH_KEYBOARD_LL), the callback function, the module handle, and the thread ID (0 = all threads).
UnhookWindowsHookEx()	Removes a previously installed hook. This cleans up the hook handle when the program is done, preventing memory leaks.
CallNextHookEx()	Passes the keyboard event to the next hook in the hook chain. This must always be called in the callback or other applications may stop receiving keyboard events.
GetMessage()	Retrieves messages from the thread's message queue. It blocks (waits) until a message is available, which keeps the program alive in the message loop.
TranslateMessage()	Translates virtual-key messages into character messages. Required for proper message handling in the Windows message loop.
DispatchMessage()	Dispatches a message to the appropriate window procedure. Works together with GetMessage and TranslateMessage.
GetAsyncKeyState()	Checks whether a specific key is currently pressed. Used in logKeystroke to detect if Shift is held down when a letter key is pressed.
GetModuleHandle()	Returns the handle of the current executable module, required when installing the keyboard hook.
GetLastError()	Returns the last error code from a Windows API call, used for debugging when SetWindowsHookEx fails.

9. Limitations and Security Concerns

9.1 Limitations of the Current Project

While the keylogger works for its intended educational purpose, there are several limitations that should be noted:

- The log file is saved as plain text (keylog.txt) and anyone who finds the file can read it easily. There is no encryption on the stored data.
- The program only handles a limited set of keys. Many special characters, function keys (F1-F12), and symbols are logged as unknown key codes like [65] instead of readable text.
- The program does not record which application was active when a key was pressed. This makes it hard to know the context of the keystrokes.
- The console window is visible on the screen. A real-world malicious keylogger would typically hide itself completely, but this project intentionally does not do that.
- The program only runs on Windows. It cannot work on Linux or macOS since it uses the Win32 API.
- There is no network functionality. The log file stays on the local machine and is not sent anywhere.

9.2 Ethical and Legal Considerations

It is critically important to understand the legal and ethical side of keyloggers. Running a keylogger on any computer without the owner's explicit knowledge and consent is illegal in most countries. This includes Pakistan, where unauthorized surveillance is a criminal offense.

This project was built only to understand how such software is coded and to fulfill the OOP course requirement. It should never be compiled, run, or distributed on any system that you do not personally own and have explicit permission to test on. The program was tested only in a controlled, personal development environment.

As cybersecurity students, we have a responsibility to use our knowledge ethically. Understanding how attacks work is necessary — but only so we can build better defenses, not to harm others.

10. Possible Improvements

If this project were to be developed further (for example in a higher semester or as a research project), the following improvements could be made:

10.1 Technical Improvements

- **Encryption:** The log file could be encrypted using AES or another cipher so that the data cannot be read without a key. This would be a good use of data security concepts.
- **Full Key Coverage:** More virtual key codes could be handled, including all function keys, numpad keys, and special characters like @, #, \$, etc.
- **Active Window Tracking:** The `GetForegroundWindow()` and `GetWindowText()` API calls could be used to log which application was active when each key was pressed. This would add much more context to the logs.
- **Timestamp Logging:** A timestamp (date and time) could be saved alongside each keystroke or group of keystrokes so that the log shows exactly when something was typed.
- **Screenshot Capture:** The program could periodically take screenshots using the `BitBlt` Windows API function and save them to the log folder for visual context.

10.2 OOP Improvements

- **Inheritance:** A base class called `Logger` could be created with virtual functions. `KeyLogger` could then inherit from it. This would allow other types of loggers (like `MouseLogger` or `NetworkLogger`) to be built from the same base.
- **Multiple Classes:** A separate `FileManager` class could handle all file operations. A `HookManager` class could manage the Windows hooks. This would make the code more modular and easier to maintain.
- **Exception Handling:** try-catch blocks could be added to handle errors gracefully instead of silently returning from functions.
- **Interface through Abstract Class:** A pure abstract class (interface) could define the contract for all logger types, making the design more flexible and extensible.

10.3 Defensive Use Cases

With the right modifications, the core technology of this project could be used for legitimate defensive purposes such as:

- Building a parental control tool that alerts parents when certain words are typed
- Creating an employee activity monitor (with consent) that logs work-related activity
- Developing an intrusion detection feature that alerts when unusual keyboard patterns are detected

11. Conclusion

This project successfully demonstrates the use of Object Oriented Programming concepts in a real and practical C++ application. The KeyLogger class was built using key OOP principles including encapsulation, abstraction, static members, and proper class design. The result is a clean, organized, and working program that captures keystrokes and logs them to a file.

Through this project, the following things were learned and applied:

- How to design and build a class in C++ with proper private and public sections
- How encapsulation protects data from being misused outside the class
- How abstraction simplifies the interface of a complex program
- How static member variables and functions solve the problem of using a class instance inside a static callback
- How to use Windows API functions to interact with the operating system at a low level
- How low-level keyboard hooks work in Windows
- How to read and write files in C++ using ofstream

From a cybersecurity perspective, this project was a valuable learning experience. Understanding how a keylogger works from the ground up gives important insight into how systems can be compromised. This knowledge is essential for a cybersecurity professional who wants to build tools that detect or prevent such threats.

The project also highlighted the ethical responsibilities that come with cybersecurity knowledge. Knowing how to build something that could be misused makes it even more important to use that knowledge responsibly, legally, and only for the purpose of improving security and protecting systems.

Overall, this was a successful and educational project that met all the goals set at the beginning. It applied OOP in a meaningful context, showed how system-level programming works in Windows, and provided a foundation for further exploration in both software development and cybersecurity.

End of Report

Object Oriented Programming | Semester 2 | Cybersecurity Department